Coupled Pendulum's Resonate

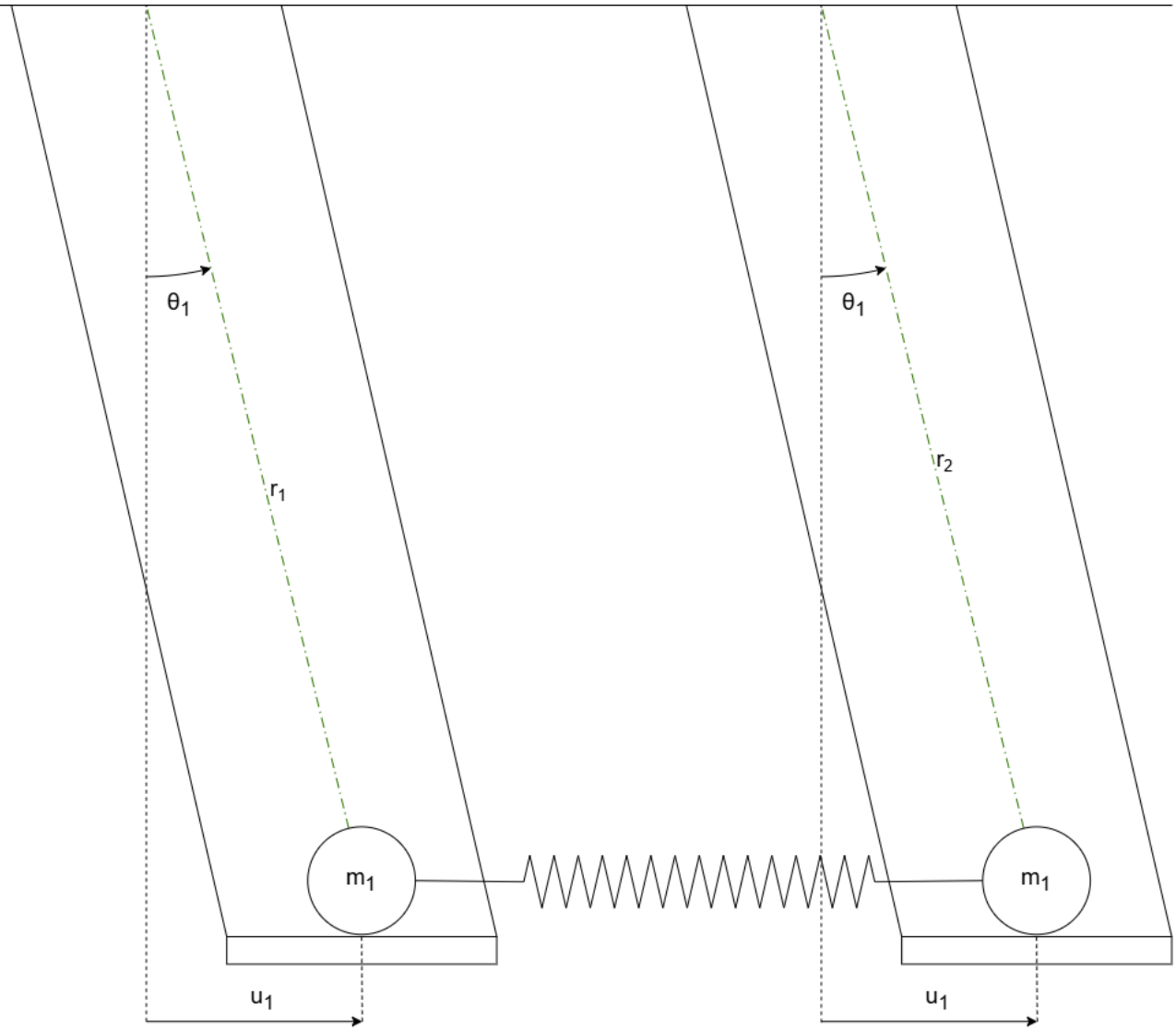
Table of Contents

Ideal Model: 2
Tridiagonal Toeplitz Matricies: 3
Real world data: 4
- Analysis tools 4
- Trial set up: 5
  - Normal Mode 1: 6
  - Normal Mode 2: 6
  - Mixed Mode: 6
- Normal Mode 1: 6
  - Trial 1: 6
  - Jitter: 7
  - Non ideal pendulum behaviour: 7
  - Trial 2: 8
- Normal Mode 2: 9
- Trial 1: 9
- Trial 2: 10
- Trial 3: 11
- Normal Mode 3 (Mixed Mode): 12

Side Quest's 13
- Calculating pi with a pendulum: 13

Important considerations: 14
- Equilibrium: 14
- Damping: 14
  - Sources: 15
  - Effects: 15
- Beat Frequencies: 16

Ideal Model:



Upon staring at your real world model for long enough you may eventualy find the the proper parameters to extract in order to turn your problem into math. Here we have identified the characteristic paramaters or, if you will, eigen-parameters, to be:

$$\vec{u_i}(t) := \text{ displacement from equilibrium}$$
$$\theta_i(t) := \text{ angle from equlibrium}$$
$$r_i := \text{ length of i'th pendulum}$$
$$m_i := \text{ mass oif the i'th pendulum}$$
$$k := \text{ spring constant}$$

$$\begin{bmatrix} \ddot{u}_1 \\ \ddot{u}_2 \end{bmatrix} = \begin{bmatrix} \frac{g}{r} + \frac{k}{m} & -\frac{k}{m} \\ -\frac{k}{m} & \frac{g}{r} + \frac{k}{m} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
clear; close; clc;

g = 9.81;
r = 1.75;
k = 6.528;
m = 3.336; %% should be 3.336
```

Main Diagonal:

```
a = g/r + k/m;
```

Super Diagonal:

```
b = -k/m;
```

Sub Diagonal:

```
c = -k/m;
```

Tridiagonal Toeplitz Matricies:

$$\begin{bmatrix} a & b & & & \\ c & a & b & & \\ & c & \ddots & \ddots & \\ & & \ddots & \ddots & b \\ & & & c & a \end{bmatrix}$$

$$a = \frac{g}{r} + \frac{k}{m}, \ b = -\frac{k}{m} \ \& \ c = -\frac{k}{m}$$

EigenValues and EigenVectors:

```
Eigen_Values = sqrt( -a + 2.*( sqrt(b*c) ).*cospi( (1:2) ./ (2 + 1) ) );

Eigen_Vectors = sinpi( ((1:2).*(1:2)')./ (2+1) );

SOLVE( 1:0.1:2, [1,1], [-1,1], a, b, c, 2 );
```

```
function U = SOLVE( varargin )
    %persistent == static
    persistent Eigen_Vectors Eigen_Values;

    tspan = varargin{1};
    u_i = varargin{2};
    v_i = varargin{3};

    % ( tspan, u_i, v_i, a, b, c )
    if( nargin == 7 )

        a = varargin{4};
        b = varargin{5};
        c = varargin{6};
        n = varargin{7};
        Eigen_Vectors = sinpi( ((1:n).*(1:n)')./ (n+1) );
        Eigen_Values = sqrt( -a + 2.*( sqrt(b*c) ).*cospi( (1:n) ./ (n + 1) ) );

    end

    C = mldivide(Eigen_Vectors , u_i');
    C = C - mldivide( Eigen_Vectors.*Eigen_Values , v_i' );

    U = real( Eigen_Vectors * ( C .* exp( (Eigen_Values' .* tspan ) ) ) );

end
```

Real world data:

Analysis tools

```
% this function, takes in a file name and a time span and returns a data
% strcture containing:
% U1, U2      : displacments of each mass
% V1, V2      : velocities of each mass( only if data file contains velocities )
% t           : time range
% freq        : aprox frequency of mass 1 from a fourier curve fit
% Modeled     : Modeled displacments of each mass based on the intial values of the
data
function DATA = Extract( filename, tspan )

    Temp = importdata( filename ).data;
    span = ( find( Temp(:,1)==tspan(1) ):find( Temp(:,1)==tspan(2) ) );
```

```
    if numel( Temp(1,:) ) == 5
        index_2 = 4;
        DATA.V1 = Temp( span, 3 );
        DATA.V2 = Temp( span, 5 );
    else
        index_2 = 3;
    end


    DATA.U1 = Temp( span, 2 );
    DATA.U1 = DATA.U1 - mean( DATA.U1 );

    DATA.U2 = Temp( span, index_2 );
    DATA.U2 = DATA.U2 - mean( DATA.U2 );

    DATA.t = Temp( span, 1 ) - tspan(1);


    DATA.freq = fit( DATA.t, DATA.U1, "fourier1" ).w;

    if( numel(Temp(1,:)) == 5 )
        DATA.Modeled = SOLVE( DATA.t', [DATA.U1(1),DATA.U2(1)],
[DATA.V1(2),DATA.V2(2)] );
    else
        DATA.Modeled = SOLVE( DATA.t', [DATA.U1(2),DATA.U2(2)],[0,0] );
    end

end

%    It's in the name, this function makes a plot formated for the above data
%    structure
function PlotData( Data, Trial )

    scatter( Data.t, [ Data.U1,Data.U2 ], 20, '.' );
    hold on
    plot( Data.t, Data.Modeled );
    hold off
    title(Trial);
    legend( ["Mass A","Mass B","Mass A Modeled","Mass B Modeled"] );


end
```

Trial set up:

In general it seems like there are two ways to approach this:

1. Set up a trial for specific initial conditions

2. Set up up trial for a specific behavior

The first method would make it easy to set up the model (assuming your initial conditions are precise enough), but this method adds extra variables to pay attention to. while the second method is laser focused on the important characterisic (the eigenvectors), and since that is the thing we are intent on validating it seems prudent to focus on that. So, I think well go that route, and we'll need different set ups to test each mode:

Normal Mode 1:

Goal: Displace both masses an equal amount

1. Jeff's ruler method: essentially make a parallelagram out of the two pendulums, and a ruler. if you make this parallelagram when the two masses are at equilibrium, then (ideally) if you shear this paralleagram, by moving the ruler and the masses to the side, the equilibrium should be preserved. The only problem with this is that we dont live in an ideal world and you will probably introduce some extenuos movement when you release the masses. ALSO if you grab the masses from the side, they will try to twist when you shear the paralleagram

Normal Mode 2:

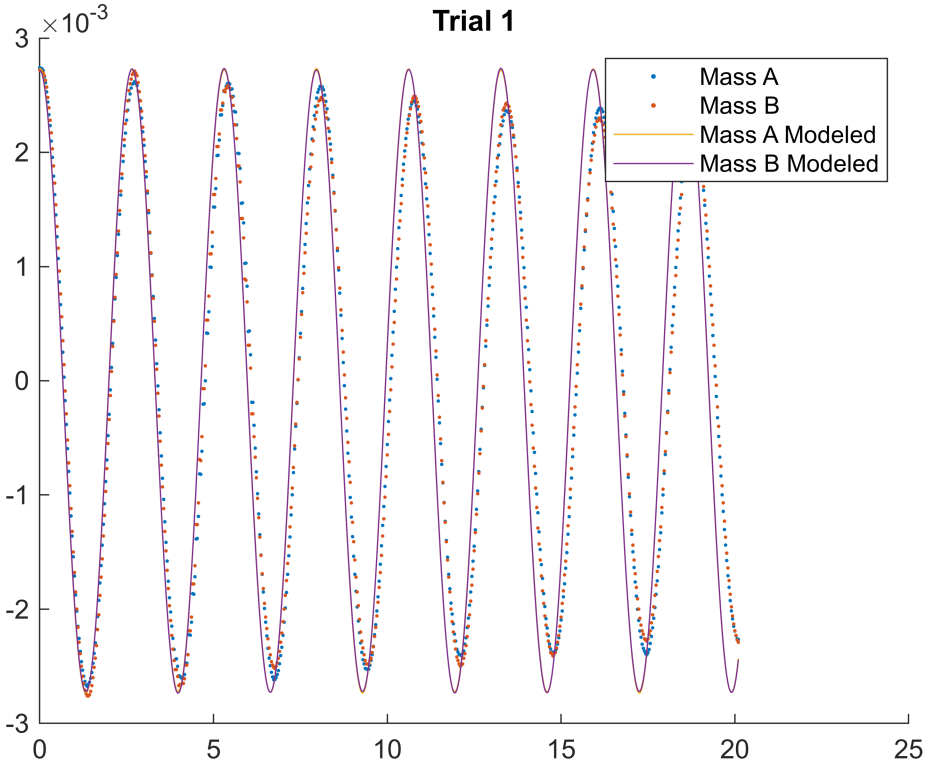
Goal: Displace both masses an equal and opposite amount

Mixed Mode:

Normal Mode 1:

Trial 1:

```
Mode_1_Trial_1 = Extract( "Normal_Mode_1_full.txt",[57.267,77.367] );
PlotData( Mode_1_Trial_1,"Trial 1" );
```




```
( Mode_1_Trial_1.freq - imag(Eigen_Values(1)) ) / imag(Eigen_Values(1)) * 100
```

```
ans =
-0.8647
```

Jitter:

There seems to be a consistent jitter in the data around 4 - 6 seconds into the replay. I think this is independant of the recording time, since the clips were chopped up after the fact, so it could be from post processing:

1. Clipmaster: the jitter could be introduced by the goofy editing sofrware used to chop up the clips
2. Tracker: the jitter could be from a poor autotrack in the tracker sofware(the jitter does seem to be caused by two datapoints esentially on top of eachother)

Non ideal pendulum behaviour:

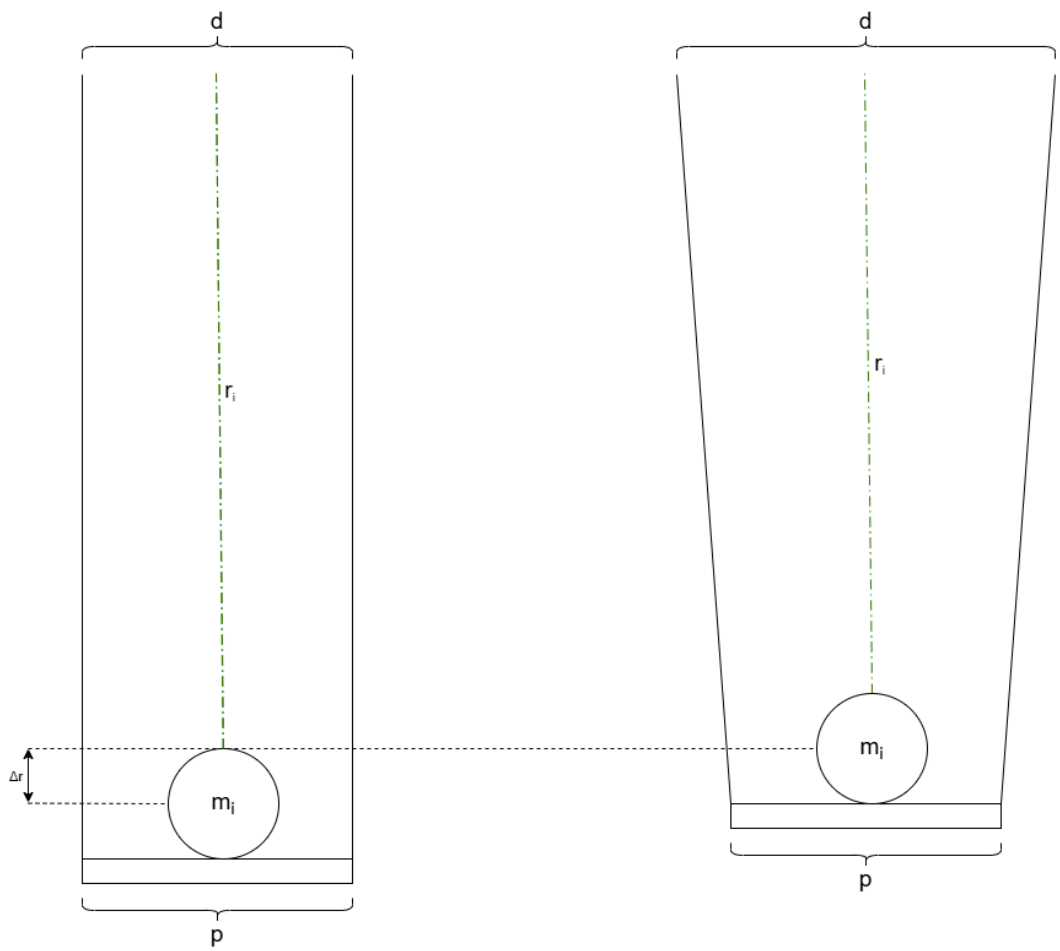
One of my collegues noticed that the real world model we're testing on doesn't neccisarily act as an ideal pendulum. Our aproximation of a pendulum is a actualy a square platform suspended from each corner by a wire that is anchored to a utility rack. These wires are splayed out on the axis perpendicular to the direction of travel and evenly spaced on the axis parallel to the direction of travel. In theory this should create a parallel linkage whose center of mass follows the trajectory of an ideal pendulum (a perfectly circular path). Even though, in practice, there really is no such thing as a perfect circle, we can assume that the slight diffferences will be negligable compared to the other sources of error in this system (measuerments of the coeffs, and displacement, etc). However, that assumption gets worse and worse the further our linkage strays from an ideal parallelagram. The question is, How much worse does this assumption get over the range of realistic values(keeping in mind our previous assumption of small angles)

Change: distance between the wires

effect: change in period

Cause:

1. Mass no longer follows an ideal circular path and cant be modeled as a pendulum. - after staring at a desmos graph for long enough it doesn't seem like the path deviates from our ideal circular path by any apreacable amount(at least while staying within the limitis of our small angle aproximation).
2. **Moments introduced by the platfrom sway take up some of the potential energy leading to a lower acceleration due to the pendulum (lower return force means longer period)**
3. *The true radius of the pendulum changed* - This doesn't line up since the radius decreases and this should cause an increase in frequency, but the observed change is a decrease in frequency



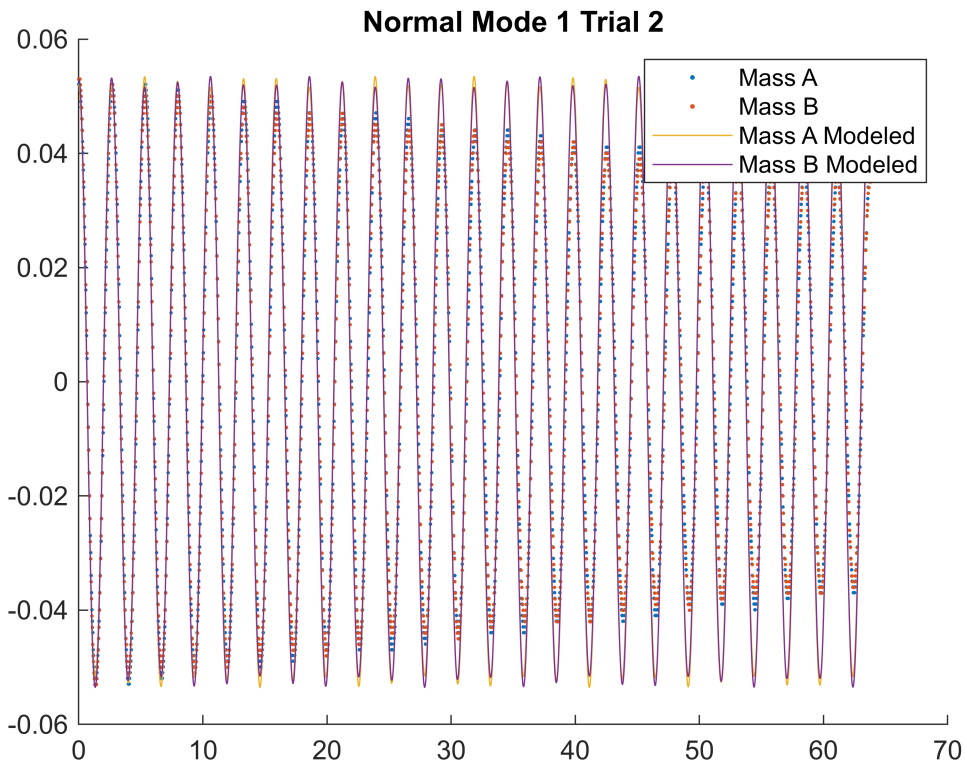
Either way, let's just try our best to make the our real world model look like the diagram on the left, and if that lines up with our theory we can move on.

Trial 2:

Changes:

- Payed extra attention to the spacing of the wires at the top of the rig. They should be spaced approx 11.8 cm appart.

```
Mode_1_Trial_2 = Extract( "Normal_Mode_1_2.txt", [0,63.8] );
PlotData( Mode_1_Trial_2, "Normal Mode 1 Trial 2" );
```



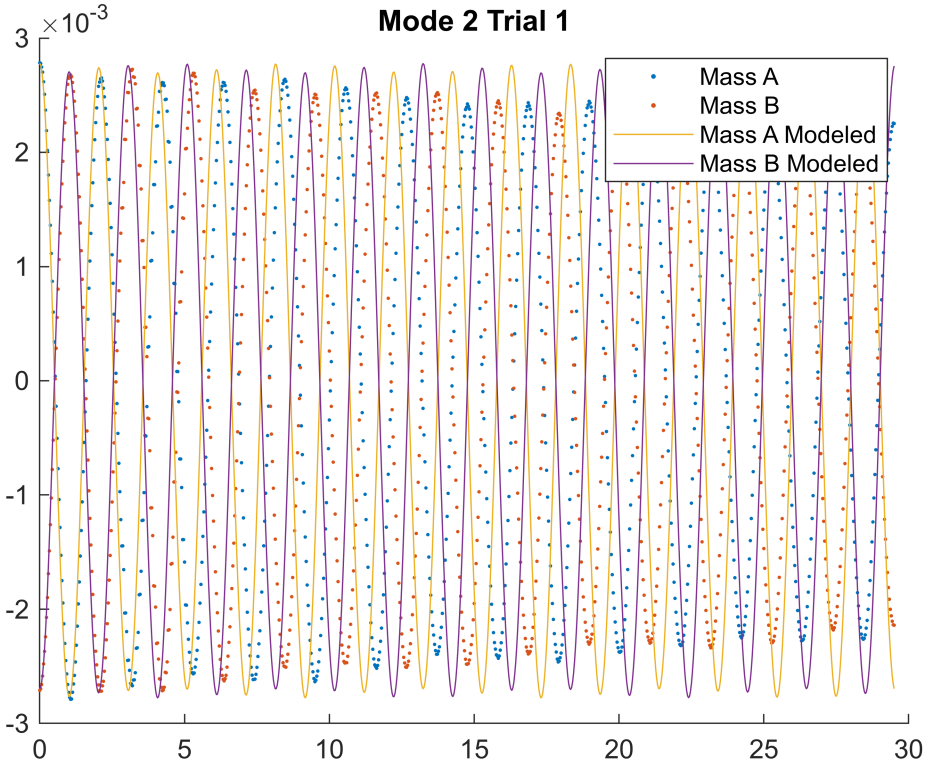

```
( Mode_1_Trial_2.freq - imag(Eigen_Values(1)) ) / imag(Eigen_Values(1)) * 100
```

```
ans =
-0.0777
```

Normal Mode 2:

Trial 1:

```
Mode_2_Trial_1 = Extract( "Ian_Moss_Normal_Mode_2.txt", [1.6,31.1] );
PlotData( Mode_2_Trial_1,"Mode 2 Trial 1" );
```




```
( Mode_2_Trial_1.freq - imag(Eigen_Values(2)) ) / imag(Eigen_Values(2)) * 100
```

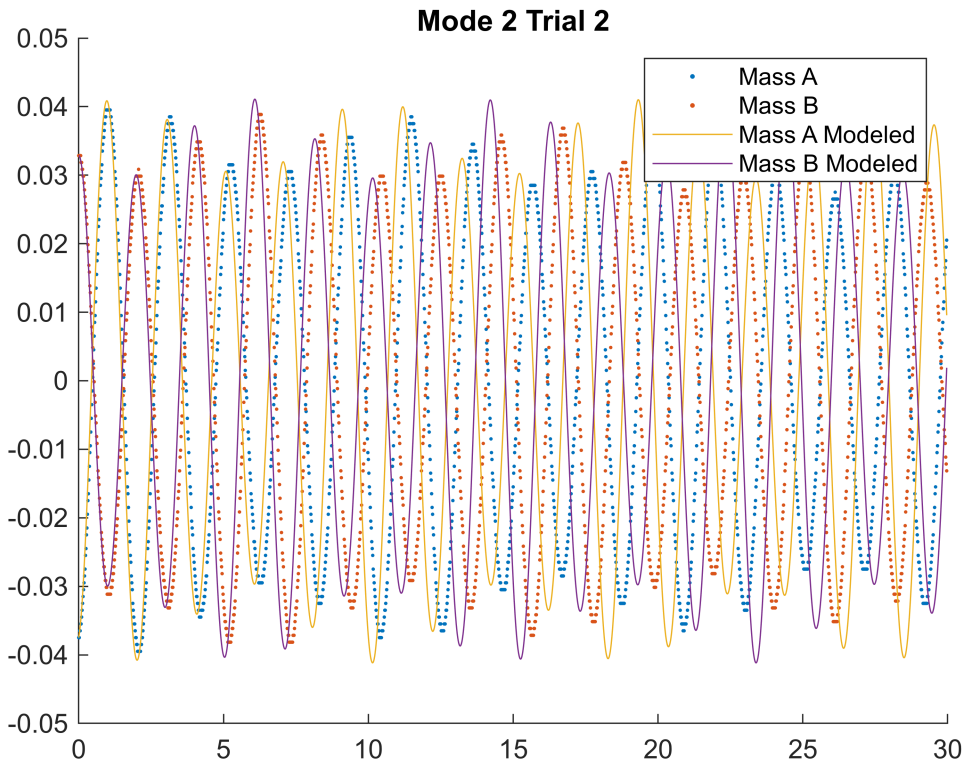
```
ans =
-3.2332
```

Trial 2:

Changes:

- Payed extra attention to the spacing of the wires at the top of the rig. They should be spaced approx 11.8 cm appart.

```
Mode_2_Trial_2 = Extract( "Mode_2_Trial_2.1.txt", [0,29.967] );
PlotData( Mode_2_Trial_2,"Mode 2 Trial 2" );
```




```
( Mode_2_Trial_2.freq - imag(Eigen_Values(2)) ) / imag(Eigen_Values(2)) * 100
```

```
ans =
-2.5152
```

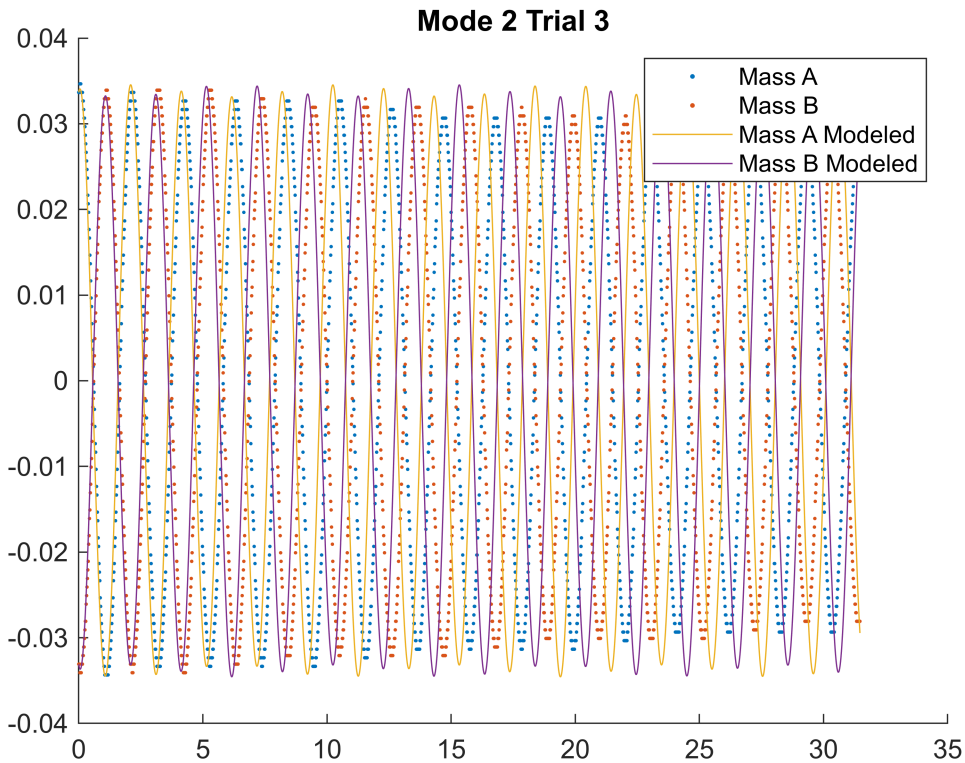
Trial 2 has a slightly better error, but it's still off by a considerable amount, maybe the pandulum legs still arent quite parallel causing the g/l affect on the frequency to be inacurate. however it seems that by changing the mass to 3.8 we can get down to a reasonable error, so this may be caused by some new error in the k/m term.

Trial 3:

Changes:

- placed the ruler between the two bolt threads on the pottom of each platform.

```
Mode_2_Trial_3 = Extract( "Mode_2_Trial_3.txt",[0,31.467] );
PlotData( Mode_2_Trial_3, "Mode 2 Trial 3" );
```

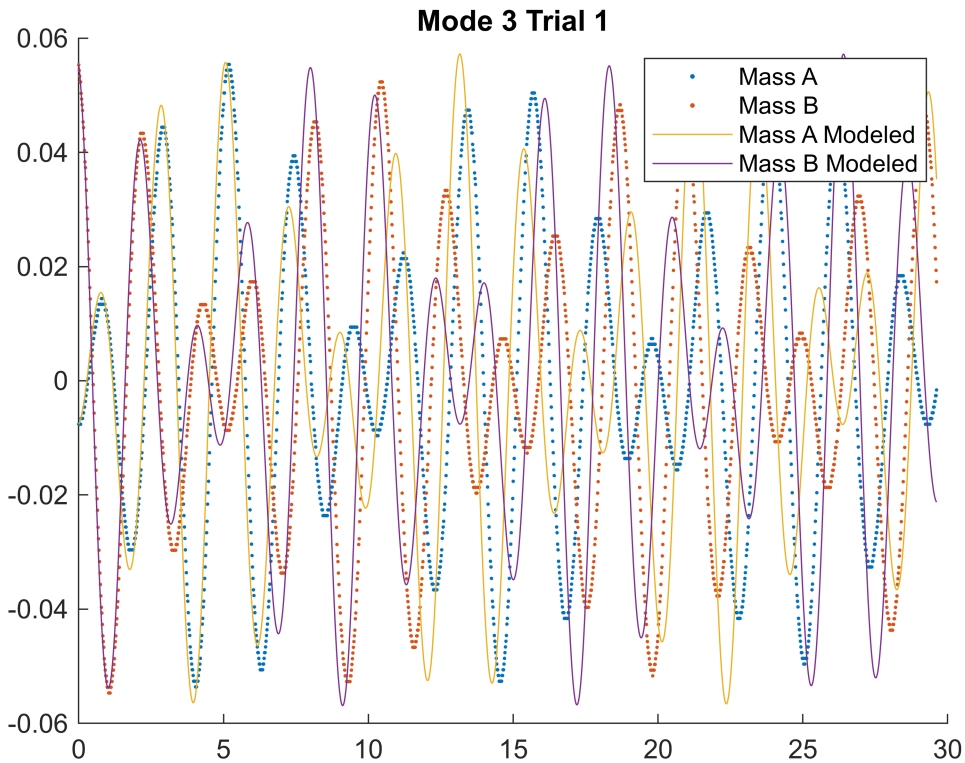



```
( Mode_2_Trial_3.freq - imag(Eigen_Values(2)) ) / imag(Eigen_Values(2)) * 100
```

```
ans =
-2.5346
```

Normal Mode 3 (Mixed Mode):

```
Mode_3 = Extract( "Mode_3_Trial_1.1.txt", [0.117,29.733] );
PlotData( Mode_3, "Mode 3 Trial 1" );
```



Side Quest's

Calculating pi with a pendulum:

Seeing as it's pi day and we have all this real world data from pendulums that we just found the complex frequencies of, I don't see why we shouldn't make a horrible approximation for pi.

For any given complex exponential function, the period and frequency can be relateded by:

$$
\begin{aligned}
e^{0i} &= 1 = e^{2\pi i} \\
\Rightarrow e^{0\omega i} &= 1 = e^{\left(\frac{2\pi}{\omega}\right)\omega i} \\
\Rightarrow \ T\omega &= 2\pi \\
\Rightarrow \pi &= \frac{T\omega}{2}
\end{aligned}
$$

Note: this relationship is a result of the math we are using to model the system, the general relationship between frequency and period is simply:

$$Tf = 1$$

It just happens to be that the the base period of a complex exponential function (which traces a unit circle in the complex plane) is 2pi. This is an important distinction because if we tried to calculate pi by experimentaly measuring both the period and frequency of a real world pendulum, well, you would end up with a very very bad approximation.

So, in order to calculate pie all we need is an experimental value for the period of our pendulum. We can achive this by just counting the occilations over a time interval and then dividing the number of occilations observed by the length of the interval. We'll use Trial 2 for Mode 1 because that is the most accurate trial so far:

```
period = Mode_1_Trial_2.t( numel( Mode_1_Trial_2.t ) ) / 24
```

```
period =
2.6583
```

```
angular_frequency = sqrt( g/r )
```

```
angular_frequency =
2.3676
```

```
my_pi = angular_frequency * period / 2
```

```
my_pi =
3.1470
```

Important considerations:

Equilibrium:

Since we define our model with respect to a displacement from the equilibrium, to be able to use this model we need a usable definition of the equilibrium.

possible solutions:

1. Record a few seconds of the masses at rest before displacing them. This would become quite cumbersome to set up, since getting a good run is essentially trial and error. (A lot of stoping and re-setting)
2. Mark the equilibrium on the background white board, to be extracted using tracker software at any time. this assumes that the equilibrium doesn't change(it really shouldn't), may also introduce extra paralax error.
3. **Or, you just take the average value over multiple FULL periods, and call that the equalibrium.**

Damping:

In our idealized model we neglected to acount for drag, assuming that drag effects would be negligable, so in order to make that assumption appropriate we need to minimize the effects drag might have on the system.

Sources:

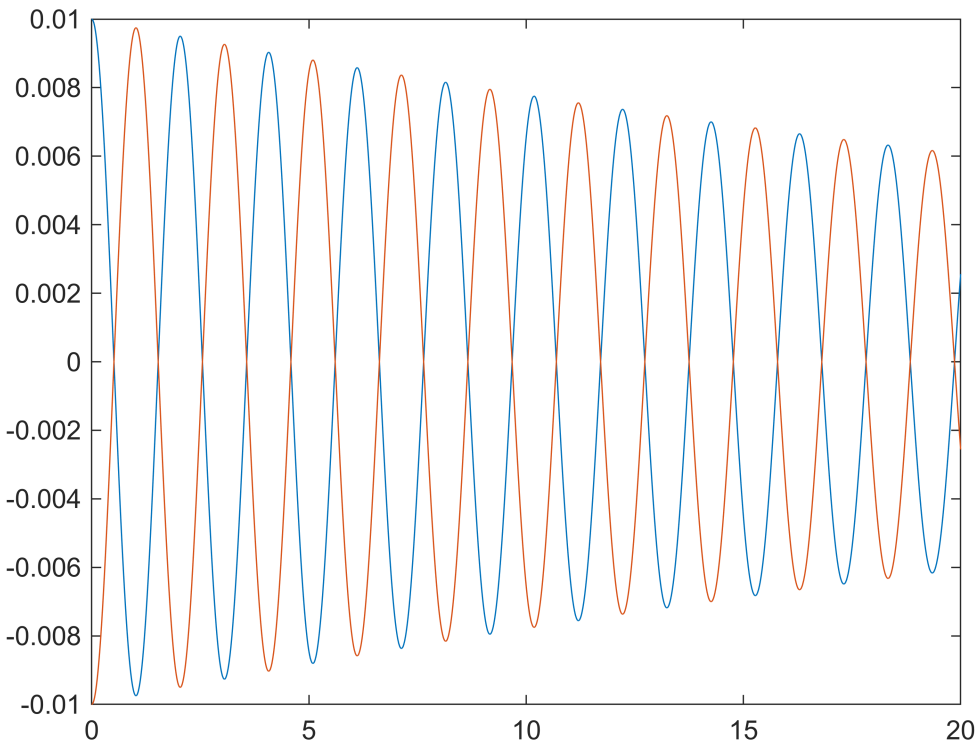
- air resistance (can't realisticly be minimized and also defininetly has little effect)
- Thermal losses in the spring (probably miniscule aswell)
- Rubbing in the pendulum wires

Effects:

The overal efffect of damping in an oscillator is to reduce the amplitude over time. However, is it possible that in a system with multiple intependent modes, that each mode is effected by the damping a different amount, leading to a change in frequency?

To test this, lets look at a sligtly modified model where we'll add a damping term proportional to the velocity of each mass.

```
b= 0.05;

B = [ 0,          1,  0,          0;
      -k/m-g/r,  -b,  k/m,        0;
      0,          0,  0,          1;
      k/m,        0,  -k/m-g/r,  -b];

          %u1;  v1;     u2;  v2
inits = [ 0.01;  0;  -0.01; 0 ];

[EigenVectors,EigenValues] = eig(B);

C = mldivide(EigenVectors , inits);

% order of operations matters!!!
A = EigenVectors * ( C .* exp( diag(EigenValues) .* (0:0.01:20) ) );

plot( (0:0.01:20), real(A([1,3],: )) );
```



Beat Frequencies: